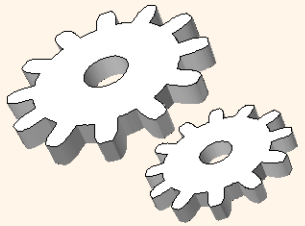


Overview of Storage and Indexing

Chapter 8

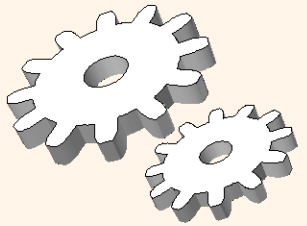
"How index-learning turns no student pale
Yet holds the eel of science by the tail."
-- Alexander Pope (1688-1744)

Data on External Storage



- Disks: Can retrieve random page at fixed cost
  - But reading several consecutive pages is much cheaper than reading them in random order
- Tapes: Can only read pages in sequence
  - Cheaper than disks; used for archival storage
- File organization: Method of arranging a file of records on external storage.
  - Record id (rid) is sufficient to physically locate record
  - Indexes are data structures that allow us to find the record ids of records with given values in index search key fields
- Architecture: Buffer manager stages pages from external storage to main memory buffer pool. File and index layers make calls to the buffer manager.

Alternative File Organizations



Many alternatives exist, *each ideal for some situations, and not so good in others:*

- Heap (random order) files: Suitable when typical access is a file scan retrieving all records.
- Sorted Files: Best if records must be retrieved in some order, or only a `range' of records is needed.
- Indexes: Data structures to organize records via trees or hashing.
  - Like sorted files, they speed up searches for a subset of records, based on values in certain ("search key") fields
  - Updates are much faster than in sorted files.

COMPARISON OF DIFFERENT FILE ORGANIZATIONS

S.No.	Sequential	Indexed	Hashed/Direct
1.	Random retrieval on primary key is impractical	Random retrieval of primary key is moderately fast.	Random retrieval of primary key is very fast.
2.	There is no wasted space for data.	No wasted space for data but there is extra space for index	Extra space for addition and deletion of records.
3.	Sequential retrieval on primary key is very fast.	Sequential retrieval on primary key is moderately fast.	Sequential retrieval of primary key is impractical
4.	Multiple key retrieval sequential file organization is possible.	Multiple key retrieval is very fast with multiple indexes.	Multiple key retrieval is not possible.
5.	Updating of records generally requires rewriting of file.	Updating of records generally requires maintenance of indexes.	Updating of records is the easiest one.
6.	Addition of new records requires rewriting the file.	Addition of new records is easy and requires maintenance of indexes.	Addition of new records is very easy.
7.	Deletion of records can create wasted space.	Deletion of records is easy if space can be allocated dynamically.	Deletion of records is very easy.

FACTORS AFFECTING CHOCE OF FILE ORGANIATION

(i) Access type : In order to search a record, whether random access or sequential access is required.
(ii) Access time : The total time taken by file organization to find a particular record.
(iii) File size : Choice is also dependent upon file size. If file size is large then choose direct access otherwise choose sequential access.
(iv) Overhead : Each technique has some overhead (it may be space overhead, time overhead etc). Any technique giving fast access may waste more space then slower techniques.
(v) Updation time : Time required to add, delete and modify any record also plays important role in efficiency.
(vi) Complexity : If technique is fast then it is complex and expensive. Whether funds are available to adopt new techniques.
(vii) Availability of hardware : The hardware that supports file organization. Example tape reader supports only sequential file organization

What is an Index?

An Index is a key built from one or more columns in the database that speeds up fetching rows from the table or view. This key helps a Database like Oracle, SQL Server, MySQL, etc. to find the row associated with key values quickly.

Two types of Indexes are:

- Clustered Index
- Non-Clustered Index

Benefits

- Prevent queries from doing full scans of an entire dataset
- Access less and therefore lock less rows during queries
- Speed up queries drastically
- Prevent sorting records post fetching
- Impose constraints e.g. data uniqueness
- Join datasets efficiently

Common problems that surface when applying indexes are

- Having an index for every column in the table
- Not utilizing composite (multicolumn) indexes
- Using composite indexes but with ineffective column orderings that prevent the index being fully utilized
- Creating indexes based on rules of thumb or heuristics, such as indexing all columns that appear in the WHERE clause

Indexes are created using a few database columns

Search Key

The first column is the **Search key** that contains a copy of the primary key or candidate key of the table. These values are stored in sorted order so that the corresponding data can be accessed quickly.

Note: The data may or may not be stored in sorted order.

Data Reference or Pointer

The second column is the **Data Reference or Pointer** which contains a set of pointers holding the address of the disk block where that particular key value can be found.

Attributes of Indexing

- **Access Types:** This refers to the type of access such as value based search, range access, etc.
- **Access Time:** It refers to the time needed to find particular data element or set of elements.
- **Insertion Time:** It refers to the time taken to find the appropriate space and insert a new data.
- **Deletion Time:** Time taken to find an item and delete it as well as update the index structure.
- **Space Overhead:** It refers to the additional space required by the index.

Types of Indexes

Primary Index

Primary index is defined on an ordered data file. The data file is ordered on akey field. The key field is generally the primary key of the relation.

Secondary Index

Secondary index may be generated from a field which is a candidate key and has a unique value in every record, or a non-key with duplicate values.

Clustering Index

Clustering index is defined on an ordered data file. The data file is ordered on a non-key field.

From the point of view of the characteristics of the index attribute:

- Primary Index
- Clustered Index
- Secondary Index

From the point of view of the number of index references to a data file:

- Dense Index
- Sparse Index

Types of file organization mechanism to store the data

Sequential File Organization or Ordered Index File

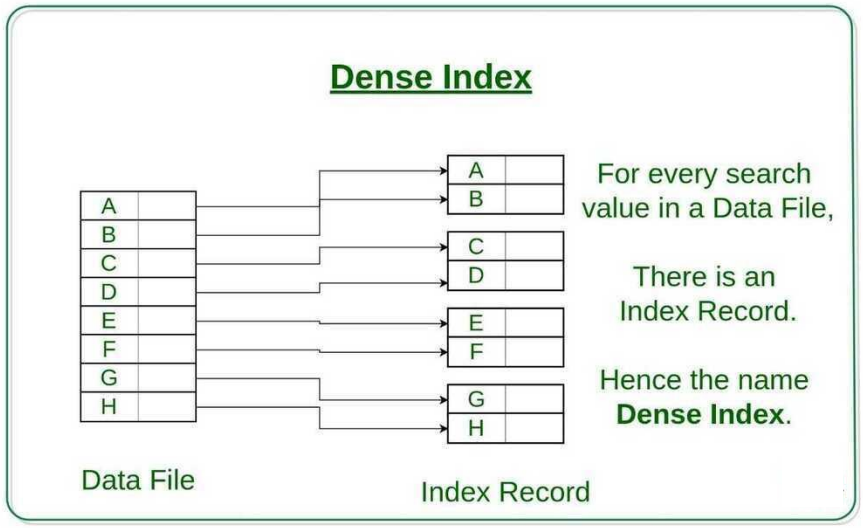
In this, the indices are based on a sorted ordering of the values. These are generally fast and a more traditional type of storing mechanism. These Ordered or Sequential file organization might store the data in a dense or sparse format:

Dense Index

•In dense index, there is an index record for every search key value in the database. This makes searching faster but requires more space to store index records itself. Index records contain search key value and a pointer to the actual record on the disk.

•For every search key value in the data file, there is an index record.

•This record contains the search key and also a reference to the first data record with that search key value.



Advantages of Dense index

(i) It is efficient technique for small and medium sized data file.
(ii) Searching is comparatively fast and efficient.

Disadvantages of Dense index

(i) Index table is large and require more memory space.
(ii) Insertion and deletion is comparatively complex.
(iii) In-efficient for large data files.

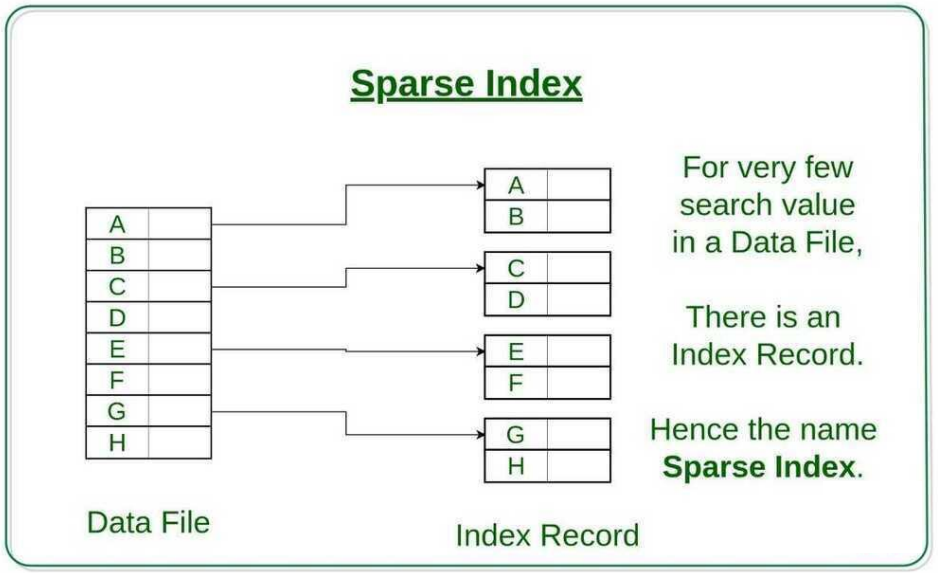
Sparse Index

•In sparse index, index records are not created for every search key. An index record here contains a search key and an actual pointer to the data on the disk. To search a record, we first proceed by index record and reach at the actual location of the data. If the data we are looking for is not where we directly reach by following the index, then the system starts sequential search until the desired data is found.

•The index record appears only for a few items in the data file. Each item points to a block as shown.

•To locate a record, we find the index record with the largest search key value less than or equal to the search key value we are looking for.

•We start at that record pointed to by the index record, and proceed along with the pointers in the file (that is, sequentially) until we find the desired record.



Advantages of sparse index

(i) Index table is small and hence save memory space (especially in large files).
(ii) Insertion and deletion is comparatively easy.

Disadvantages of sparse index

(i) Searching is comparatively slower, since index table is searched and then linear search is performed inside secondary memory.

Primary index: A primary index consists of all prime-key attributes of a table and a pointer to physical memory
address of the record of data file. To retrieve a record on the basis of all primary key attributes, primary index is used for fast searching. Binary search is done on index table and then directly retrieve that record from physical memory. It may be sparse.

Advantages of Primary index

(i) Search operation is very fast.
(ii) Index table record is usually smaller.
(iii)) A Primary index is guaranteed not to duplicate.

Disadvantages of Primary index

(i) There is only one primary index of a table. To search a record on less than all prime-key attributes, linear search is performed on index table.
(ii) To create a primary index of an existing table, record should be in some sequential order otherwise database is required to be adjusted.

Secondary index : A secondary index provides a secondary means of accessing a data file. A secondary index may be on a candidate key field or on non-prime key attributes of a table. To retrieve a record on the basis of non-prime key attributes, secondary index can be used for fast searching. Secondary index must be dense with a index entry for every search key value and a pointer to every record in a file.

Advantages of secondary index

(i) Improve search time if search on non-prime key attributes.

(ii) A data file can have more than one secondary index.

Disadvantages of Secondary index

(i) A secondary index usually needs more storage space.

(ii) Search time is more than primary index.

(iii) They impose a significant overhead on the modification of database.

What is a Clustered index?

Cluster index is a type of index which sorts the data rows in the table on their key values. In the Database, there is only one clustered index per table.

A clustered index defines the order in which data is stored in the table which can be sorted in only one way. So, there can be an only a single clustered index for every table. In an RDBMS, usually, the primary key allows you to create a clustered index based on that specific column.

Characteristics of Clustered Index

- Default and sorted data storage
- Use just one or more than one columns for an index
- Helps you to store Data and index together
- Fragmentation
- Operations
- Clustered index scan and index seek
- Key Lookup

Advantages of Clustered Index

The pros/benefits of the clustered index are:

- Clustered indexes are an ideal option for range or group by with max, min, count type queries
- In this type of index, a search can go straight to a specific point in data so that you can keep reading sequentially from there.
- Clustered index method uses location mechanism to locate index entry at the start of a range.
- It is an effective method for range searches when a range of search key values is requested.
- Helps you to minimize page transfers and maximize the cache hits.

Disadvantages of Clustered Index

Here, are cons/drawbacks of using clustered index:

- Lots of inserts in non-sequential order
- A clustered index creates lots of constant page splits, which includes data page as well as index pages.
- Extra work for SQL for inserts, updates, and deletes.
- A clustered index takes longer time to update records when the fields in the clustered index are changed.
- The leaf nodes mostly contain data pages in the clustered index.

What is Non-clustered index?

A Non-clustered index stores the data at one location and indices at another location. The index contains pointers to the location of that data. A single table can have many non-clustered indexes as an index in the non-clustered index is stored in different places.

For example, a book can have more than one index, one at the beginning which displays the contents of a book unit wise while the second index shows the index of terms in alphabetical order.

A non-clustering index is defined in the non-ordering field of the table. This type of indexing method helps you to improve the performance of queries that use keys which are not assigned as a primary key. A non-clustered index allows you to add a unique key for a table.

Characteristics of Non-clustered Indexes

- Store key values only
- Pointers to Heap/Clustered Index rows
- Allows Secondary data access
- Bridge to the data
- Operations of Index Scan and Index Seek
- You can create a nonclustered index for a table or view
- Every index row in the nonclustered index stores the nonclustered key value and a row locator

Advantages of Non-clustered index

Pros of using non-clustered index are:

- A non-clustering index helps you to retrieves data quickly from the database table.
- Helps you to avoid the overhead cost associated with the clustered index
- A table may have multiple non-clustered indexes in RDBMS. So, it can be used to create more than one index.

Disadvantages of Non-clustered index

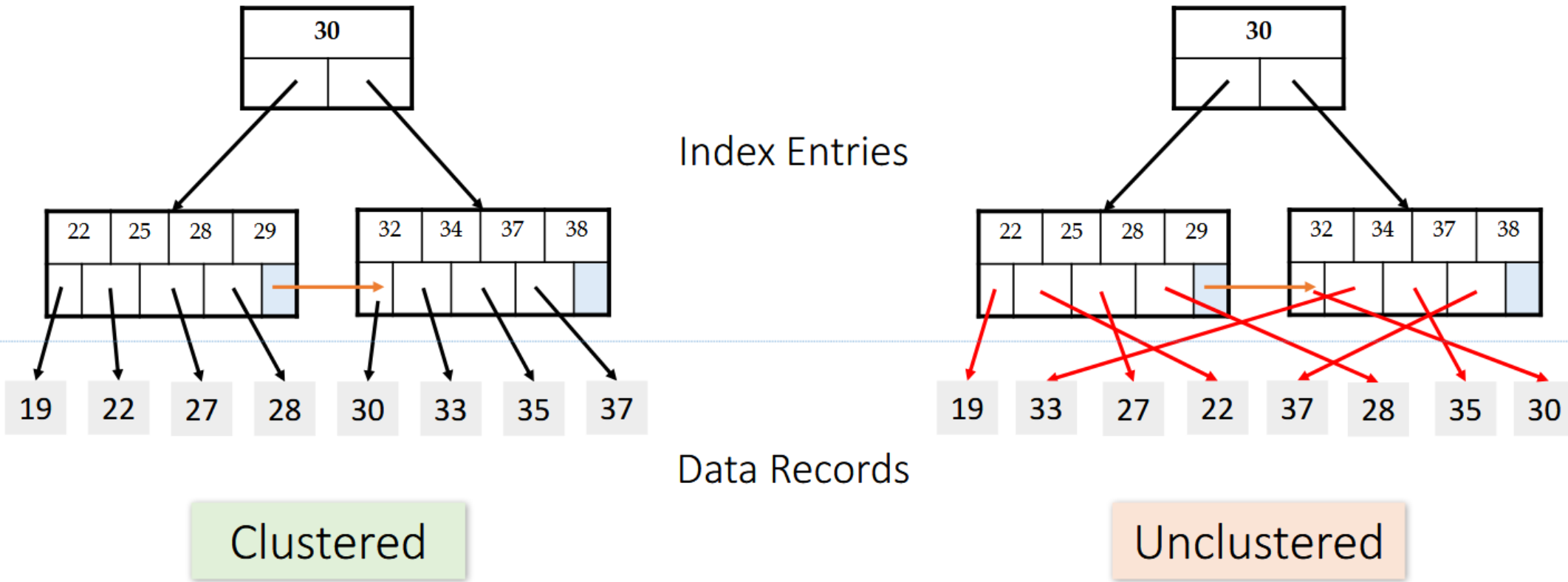
Here, are cons/drawbacks of using non-clustered index:

- A non-clustered index helps you to stores data in a logical order but does not allow to sort data rows physically.
- Lookup process on non-clustered index becomes costly.
- Every time the clustering key is updated, a corresponding update is required on the non-clustered index as it stores the clustering key.

Clustered vs Non-clustered Index in SQL: Key Differences

Parameters	Clustered	Non-clustered
Use for	You can sort the records and store clustered index physically in memory as per the order.	A non-clustered index helps you to creates a logical order for data rows and uses pointers for physical data files.
Storing method	Allows you to stores data pages in the leaf nodes of the index.	This indexing method never stores data pages in the leaf nodes of the index.
Size	The size of the clustered index is quite large.	The size of the non-clustered index is small compared to the clustered index.
Data accessing	Faster	Slower compared to the clustered index
Additional disk space	Not Required	Required to store the index separately
Type of key	By Default Primary Keys Of The Table is a Clustered Index.	It can be used with unique constraint on the table which acts as a composite key.
Main feature	A clustered index can improve the performance of data retrieval.	It should be created on columns which are used in joins.

Clustered vs. Unclustered Index



You have OLTP data that's used only for transactional reads and writing new rows. You know the primary key is an identity integer column. What type of index would you create for the primary key?

Clustered index – Your queries are probably always going to be looking up by PK to return data. If you store the data in the table ordered by that PK, SQL will be able to do this very quickly. New row additions to the table will always get put at the end because of the auto-incrementing identity column, not creating any overhead for having to insert data in a specific location in the ordered data.

You have a table that is constantly having values updated. These updated values are used as in your JOINs and WHERE clauses. What type of index would you add?

Nonclustered index – If our values are constantly changing, SQL only has to update the index and pointers while putting the actual data wherever it has available space on disk. Compare this to a clustered index where it has to put the inserted/updated data in the correct order, meaning potentially lots of operations to shift the data around if available free space doesn't exist at that location.

You have a query that wants to return most or all of the columns from a table. What type of index would make this the most efficient?

Clustered index – Since all of the column values are stored in the same location as your indexed fields, SQL won't have to go do any additional lookups to get all of the data you are requesting from it. If you created a nonclustered index you would have to INCLUDE all nonindexed columns, which would take up lots of space since you are essentially duplicating your entire table's data.

You have a table that already has a clustered index, but it doesn't cover columns in JOINs and WHERE clauses. What do you do?

Nonclustered index – since the clustered index already exists, your only option is to add a nonclustered index. Depending on the queries hitting this table however, you may want to consider changing your clustered index to a nonclustered index if you think your JOINs and WHERE clauses will be improved by having those fields be part of the clustered index.

You have a small staging table that you will always read all rows from and then truncate. You don't care about the order. Do you add an index?

No, leave it as a heap – This is one scenario where not adding an index can give you better performance since there is no overhead in SQL having to store things in a sorted order or update indexes to specify the order. If you truly don't care about the order, and you will always be reading all rows from a table and then truncating the table, then it's better not to have the overhead of having indexes on the table.

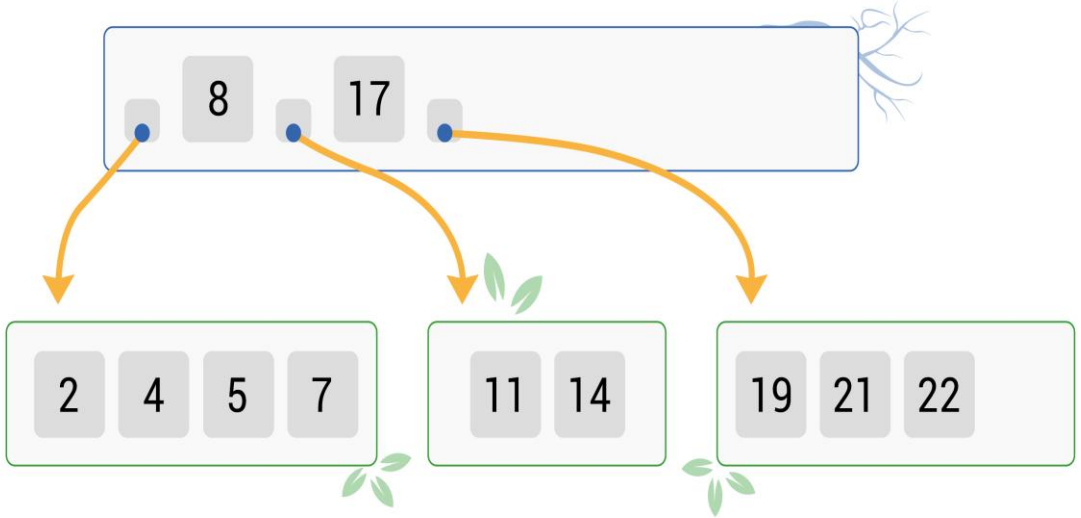
What Data Structure Does an Index Use?

As mentioned before, indexes are implemented on top of other data structures to optimize search operations. But what are these data structures?

Balanced Tree

The most common indexes use a balanced tree behind the scenes to speed up queries. Most database engines use either a balanced tree or a variation of a balanced tree like a b+ tree. The structure of a general balanced tree is

The top node is the root, and those below it are either child nodes or leaf nodes. We always start searching for our row from the root node and compare if the value we're searching for is less than or greater than the value in the node at hand. The result of the comparison tells us which way to go, left or right, depending on the result of our comparison. In the example above, all values lower than 8 take us to the left, while values greater than 8 take us to the right, and so on.

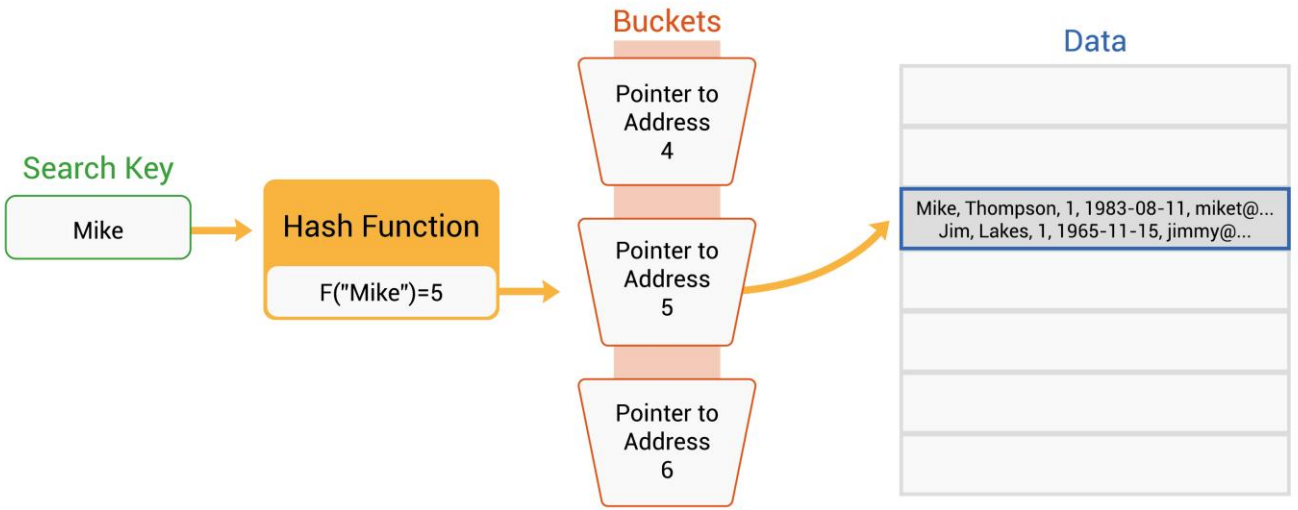


Hash

A hash is used by hash indexes. It is a data structure that provides some of the fastest search speeds. Hashes allow the index to do very fast lookups on the data stored in a table.

The idea behind hashes is that the search key is taken through a hashing function instead of being searched through an index or the entire table. The input search key is converted to a hash value that determines the associated bucket. In the example below, the search key “Mike” is taken through the hash function and is associated with a bucket.

Each bucket in the array of buckets contains the same number of records. No matter how many distinct values exist in a column, every row is mapped to one single bucket. The matching row is selected and returned from that bucket.



B Tree and B+ Tree

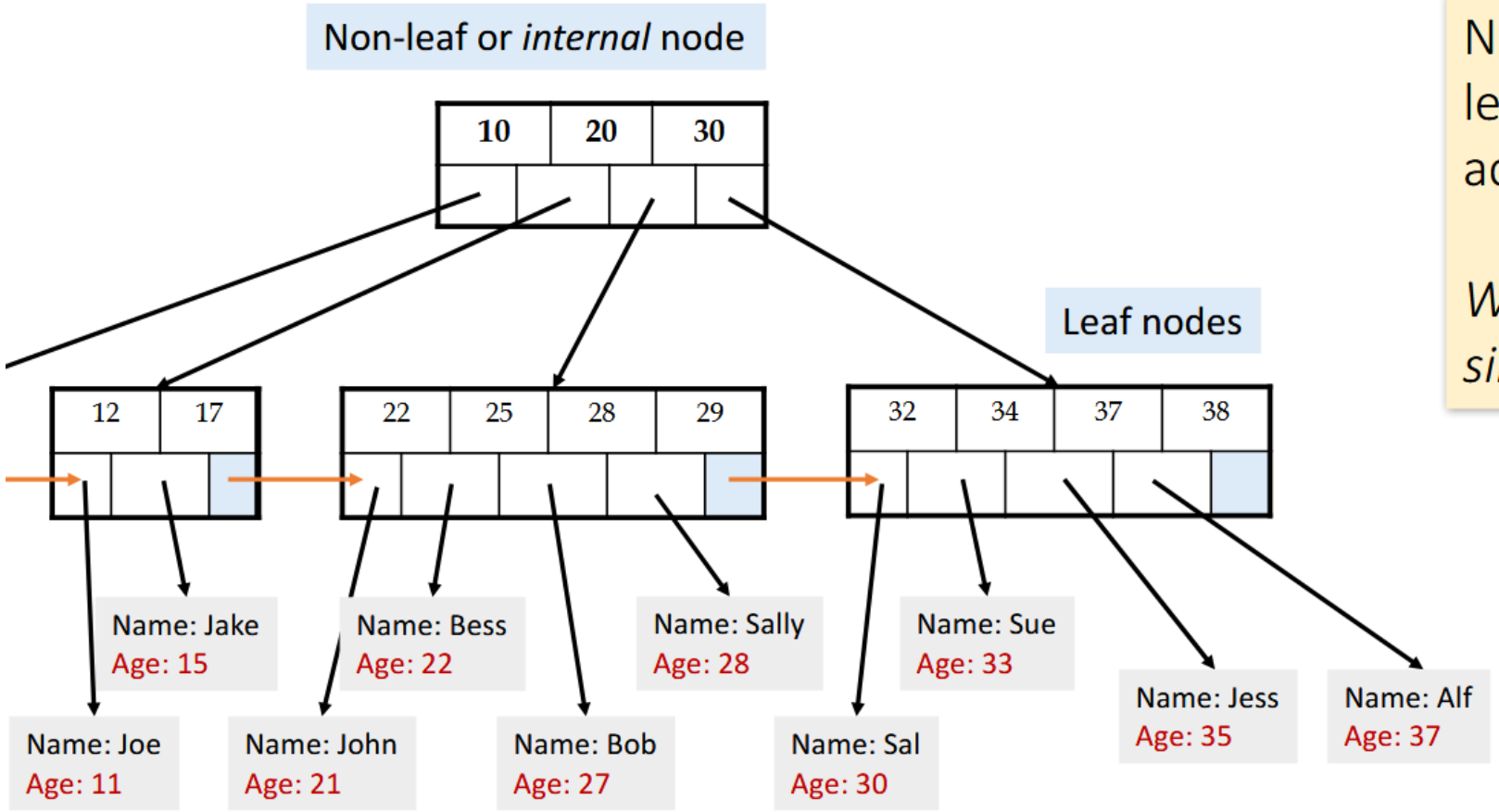
Difference between B-Tree and B+ trees

Feature	B-Tree	B+ Tree
Node Structure	All nodes, including leaf nodes, can store both keys and data. Internal nodes also contain pointers to their child nodes.	Only leaf nodes store data, while internal nodes only contain keys and pointers to child nodes.
Linked Leaf Nodes	Leaf nodes are not linked to each other.	Leaf nodes are linked together in a doubly linked list, allowing efficient range searches.
Key Distribution	Keys are distributed throughout all nodes, including internal nodes.	Keys are only stored in leaf nodes, allowing for a more balanced distribution of keys across the tree.
Search Performance	Search requires traversing the tree from the root to the leaf node containing the desired key.	Search only involves traversing the leaf nodes using the linked list, leading to potentially faster search performance for range searches.
Insertion and Deletion	Insertion and deletion can occur in both internal and leaf nodes, requiring splitting or merging nodes to maintain balance.	Insertion and deletion occur only in leaf nodes, simplifying the process and potentially reducing overhead.
Space Utilization	Internal nodes store both keys and data, leading to lower space utilization in large trees.	Internal nodes only store keys, allowing for more efficient space utilization, especially in memory-constrained environments.
Write Performance	Frequent insertions and deletions can lead to more frequent node splits and merges, potentially impacting write performance.	Localized insertions and deletions in leaf nodes can improve write performance compared to B-trees.
Applications	Suitable for applications where data access patterns are random and range searches are not crucial.	Ideal for applications requiring efficient range searches, such as databases, file systems, and search engines.

B+ Tree Basics

- Each non-leaf ("interior") node has d ≤ m ≤ 2d entries
  - Minimum 50% occupancy
- Parameter d = the order
- Root node has 1 ≤ m ≤ 2d entries
- The n entries in a node define n+1 ranges
- For each range, in a non-leaf node, there is a pointer to another node with entries in that range
- Leaf nodes also have between d and 2d entries, and are different in that:
  - Their entry slots contain pointers to data records
  - Their entry slots contain pointers to data records
  - They contain a pointer to the next leaf node as well, for faster sequential traversal

B+ Tree Basics



Note that the pointers at the leaf level will be to the actual data records (rows).

We might truncate these for simpler display (as before)…

Height = 1

Searching a B+ Tree

- For exact key values:
  - Start at the root
  - Proceed down, to the leaf

```
SELECT name
FROM people
WHERE age = 25
```

- For range queries:
  - As above
  - Then sequential traversal

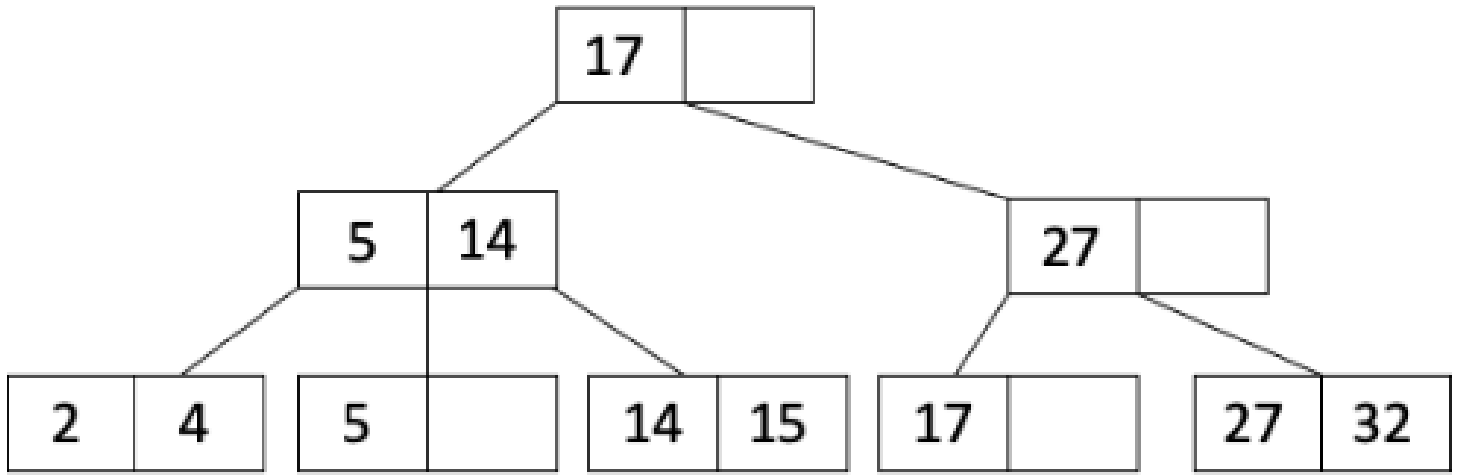
```
SELECT name
FROM people
WHERE 20 <= age
AND age <= 30
```

Pros of B+ Tree

- In an equal number of disc accesses, records can be retrieved.
- In comparison to the B tree, the B+ Tree's height stays balanced and is lower.
- The stored data in a B+ Tree can be accessed both sequentially and directly.
- Indexing is done via keys.
- Because the data is only stored on the leaf nodes, search queries are faster.

B+ Tree: Insert

- Find correct leaf L.
- Put data entry onto L.
  - If L has enough space, done!
  - Else, must split L (into L and a new node L2)
    - Redistribute entries evenly, copy up middle key.
    - Insert index entry pointing to L2 into parent of L.
- This can happen recursively
  - To split non-leaf node, redistribute entries evenly, but pushing up the middle key. (Contrast with leaf splits.)
- Splits “grow” tree; root split increases height.
  - Tree growth: gets wider or one level taller at top.


17
5
14
27
2
4
5
14
15
17
27
32

Insertion

To insert an entry into the B+ tree, follow this procedure:

(1) Find the leaf node L in which you will insert your value. You can do this by traversing down the tree. Add the key and the record to the leaf node in order.

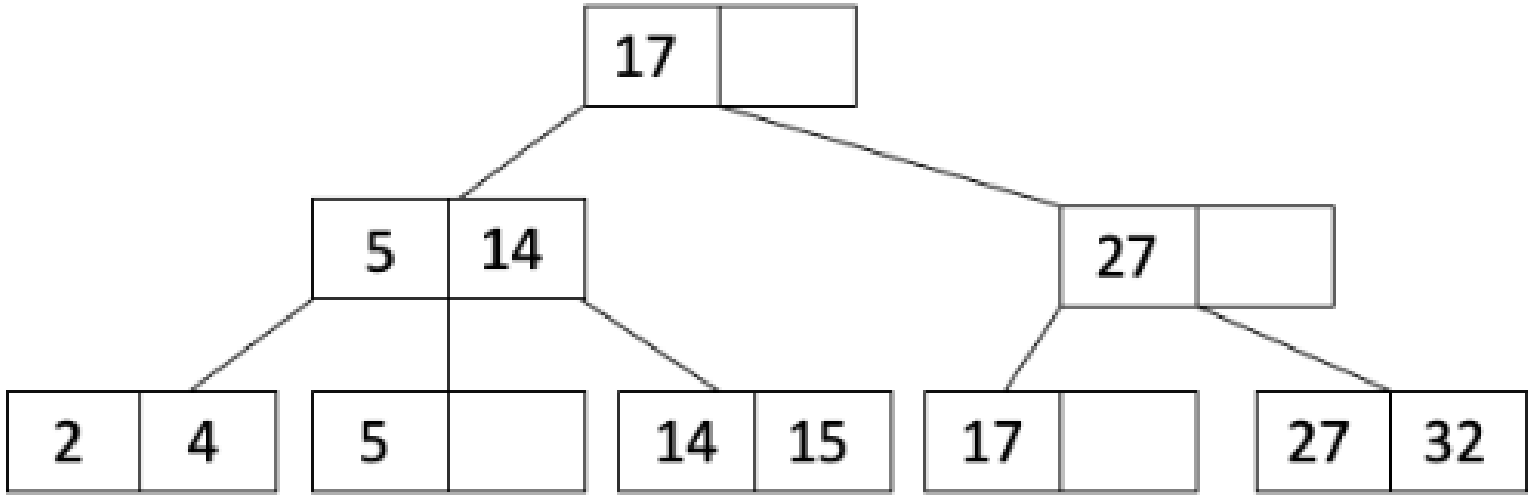
(2) If L overflows (L has more than 2d entries)...

(a) Split into L1 and L2. Keep d entries in L1 (this means d + 1 entries will go in L2).

(b) If L was a leaf node, COPY L2's first entry into the parent. If L was not a leaf node,
MOVE L2's first entry into the parent.

(c) Adjust pointers.

(3) If the parent overflows, then recurse on it by doing step 2 on the parent.

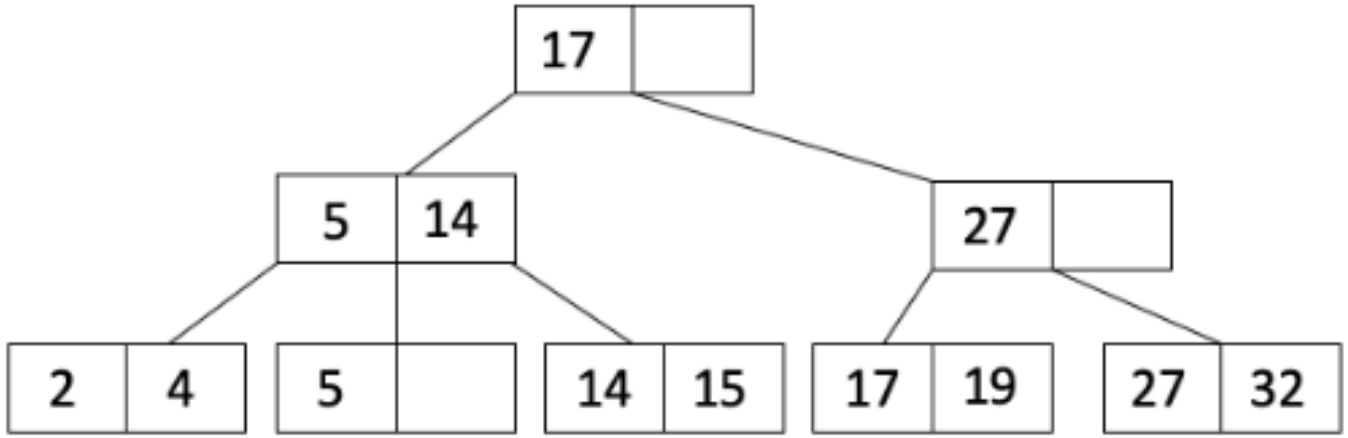


D (Order) = 1

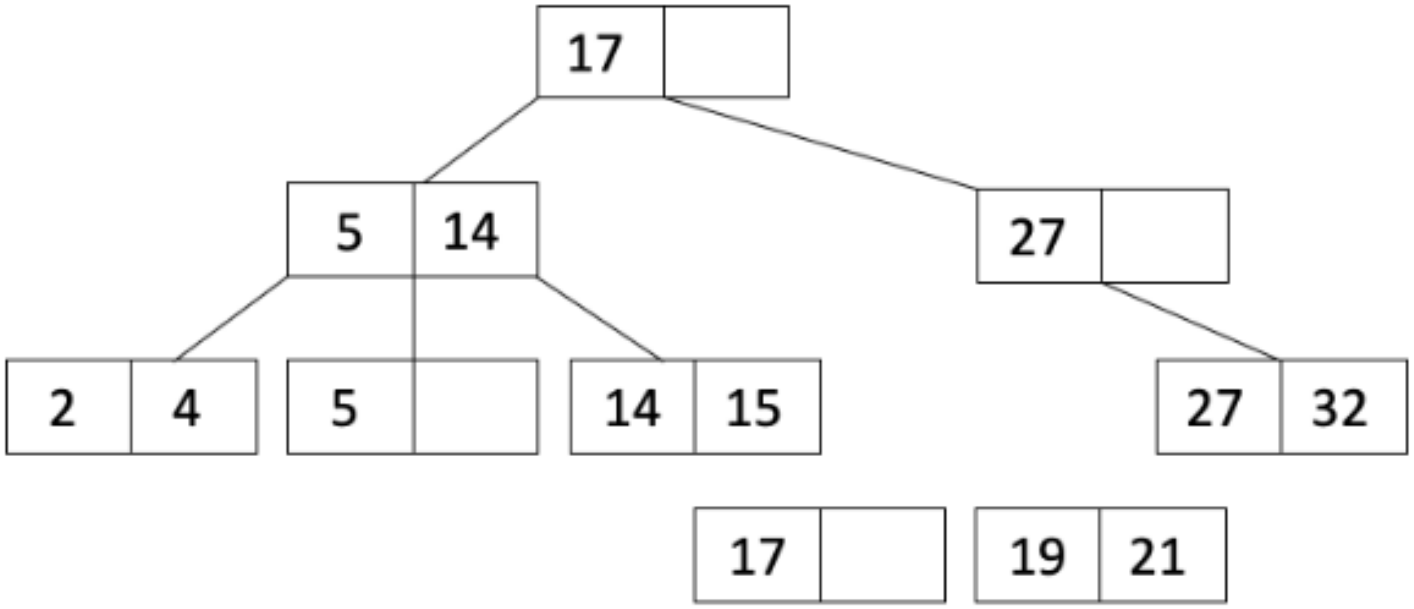
Max elements = 2

Min elements = 1

Let’s insert 19 into our tree. When we insert 19, we see that there is space in the leaf node with 17:

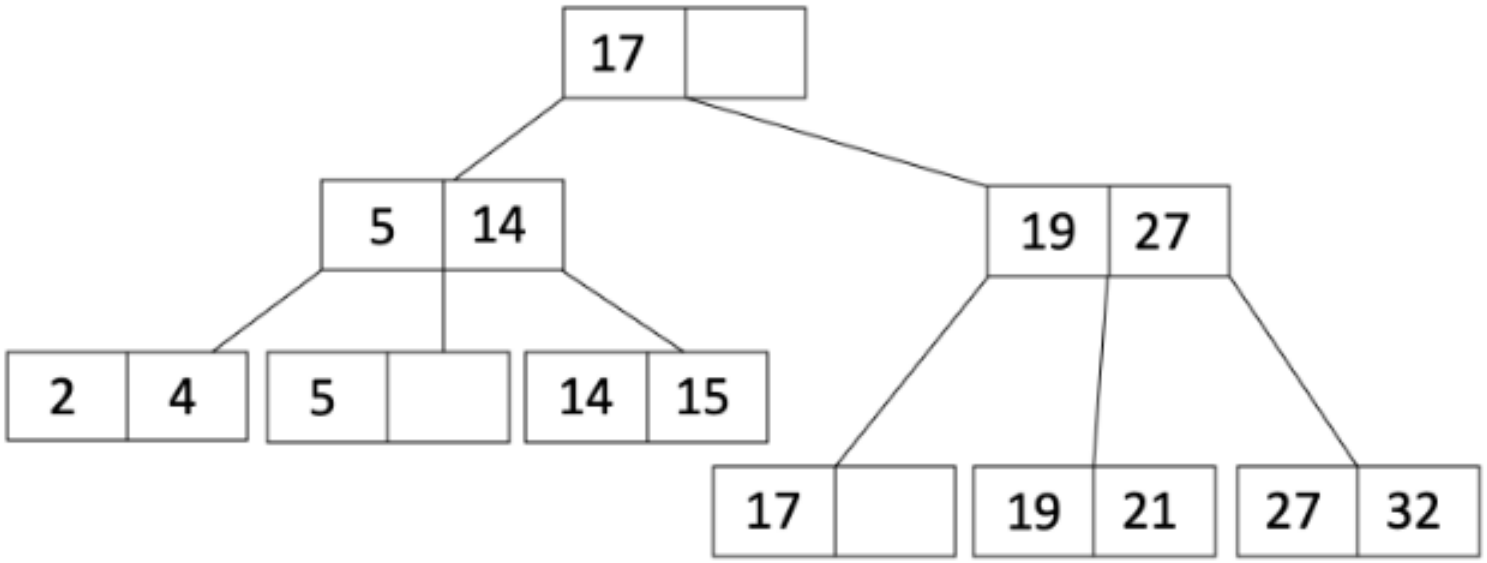


Now let’s insert 21 into our tree. When we insert 21, it causes one of the leaf nodes to overflow.
Therefore, we split this leaf node into two leaf nodes

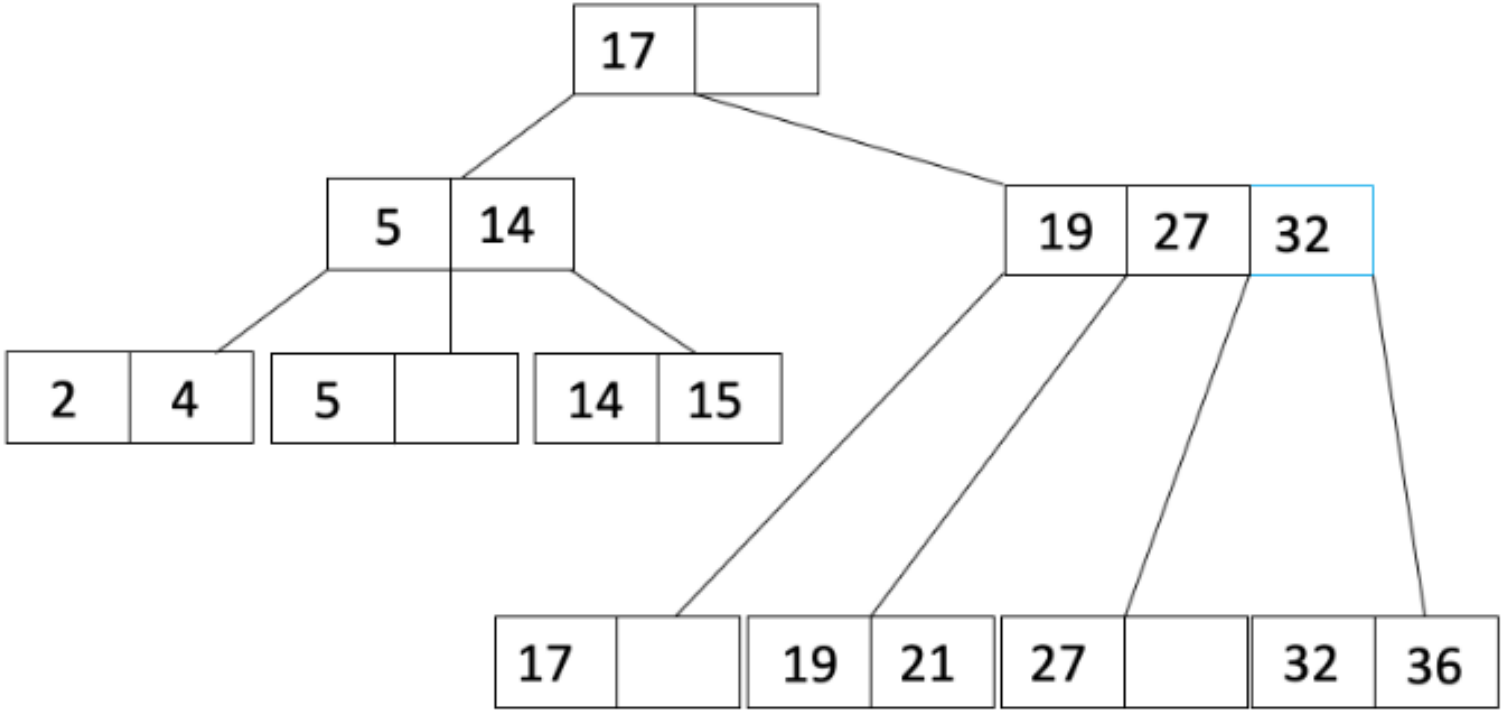


L1 is the leaf node with 17, and L2 is the leaf node with 19 and 21.

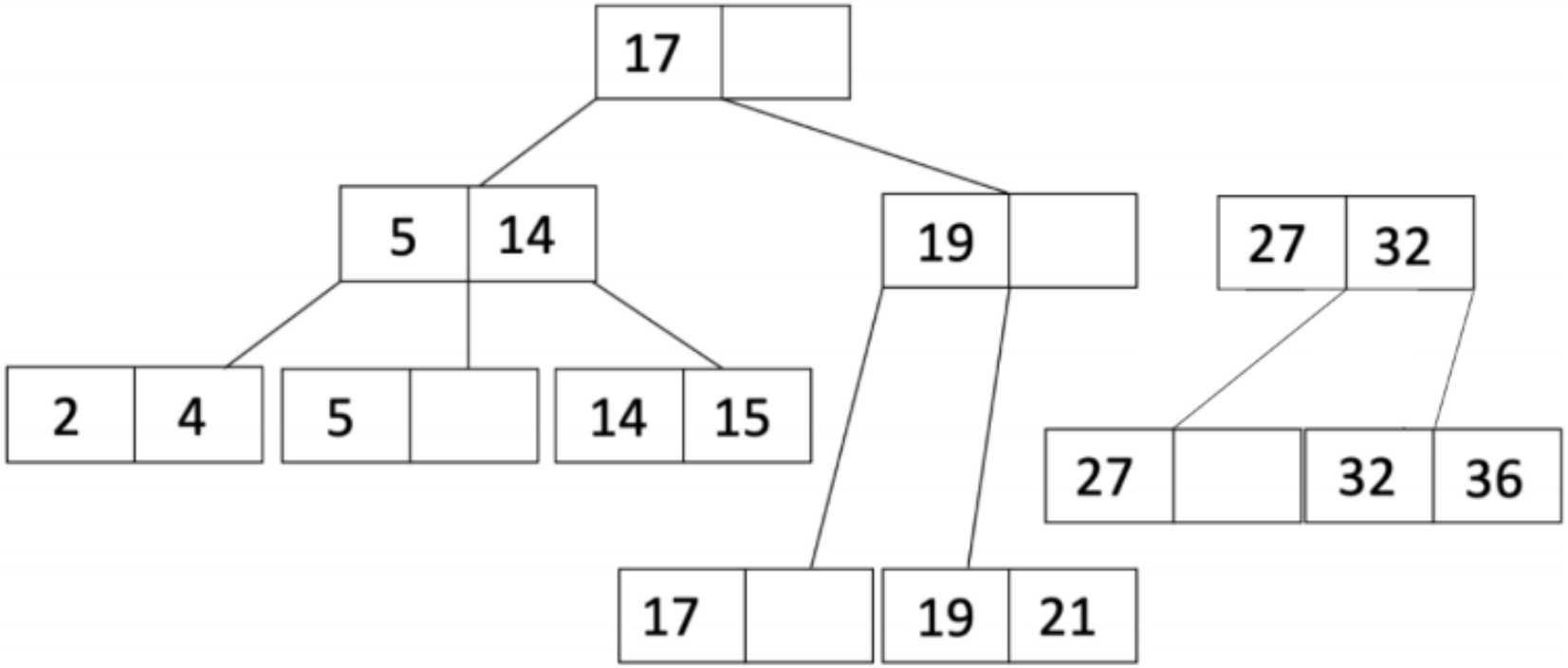
Since we split a leaf node, we will COPY L2's first entry up to the parent and adjust pointers. We also sort the entries of the parent to get:



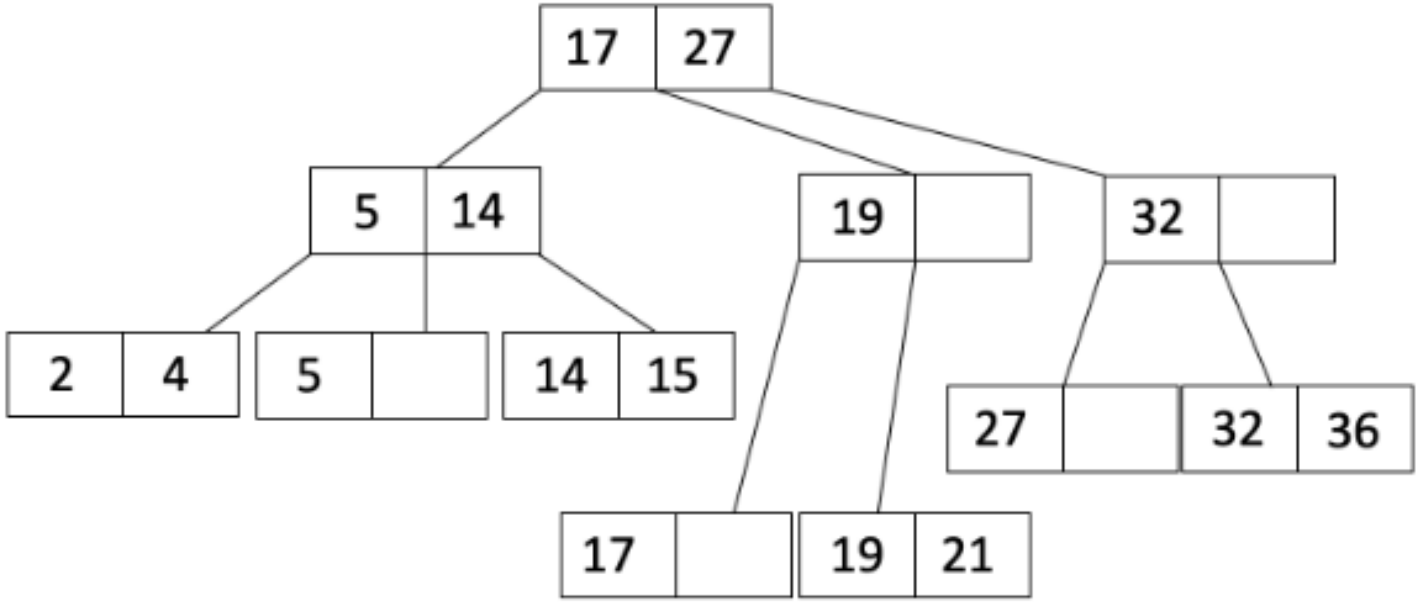
We will insert 36. When we insert 36, the leaf overflows so we will do the same procedure as when we inserted 21 to get:



Notice that now the parent node overflowed, so now we must recurse. We will split the parent node to get:



L1 is the inner node with 19, and L2 is inner the node with 27 and 32. Since it was an inner node that overflowed, we will MOVE L2's first entry up to the parent and adjust pointers to get:



B+ Tree: Deleting a data entry

- Start at root, find leaf L where entry belongs.
- Remove the entry.
    - If L is at least half-full, done!
    - If L has only d-1 entries,
        - Try to re-distribute, borrowing from sibling (adjacent node with same parent as L).
        - If re-distribution fails, merge L and sibling.
- If merge occurred, must delete entry (pointing to L or sibling) from parent of L.
- Merge could propagate to root, decreasing height.